

Stream Decoding Guide

A dedicated monograph on the reverse half of the DataShield codec — the pipeline decoding of an FEC stream into reception accumulators. The document complements `AlgorithmGuide.en.md` (sections 5–9 give an overview): here the same topic is treated formally — the problem statement, the damage model, the pipeline architecture and its invariants, the forward-pass and targeted-rebinding algorithms with completeness justifications, cost estimates, and the exact boundaries of the guarantees. An engineering retelling with matching section numbering is `DecoderGuide.Engineer.en.md`; a from-scratch tutorial is `DecoderGuide.Tutorial.en.md`. The forward transformation is the `EncoderGuide.*` family; the arbitration of the result is the `AssemblyGuide.*` family.

1. Problem statement

The input is a byte sequence `x` of arbitrary structure: a subsequence of an FEC stream's packets interleaved with arbitrary garbage; packet boundaries are unmarked. Construct a set of reception slots `Σ` `= {σf}` — one per file `f` represented by recognizable packets — such that:

1. **Accuracy.** Only packets cryptographically bound to the file's header enter the slot (false acceptance $\leq 2^{-72}$ per sector, 2^{-192} per header).
2. **Completeness.** Any undamaged packet `p ∈ x` whose membership check is decidable (a header — always; a sector — given the file's header is known at the moment `p` is reached, or after rebinding) is accepted at least once.
3. **No duplication.** A version's confirmation count is proportional to the number of physical copies of the packet in `x`, not to the number of the decoder's passes.

The output of assembling from a slot is the subject of `AssemblyGuide.Academic.en.md`; assembly is not considered here.

2. Damage model

The channel distorts the input in four ways: (a) interval erasure; (b) garbage insertion of arbitrary alphabet; (c) boundary splicing/desynchronization; (d) packet substitution. The pipeline's answers: (a) — the encoder's redundancy (ECC, header repetition) plus recovery at assembly; (b) — the alphabet filter (txt) and the byte-wise window shift; (c) — the sliding window with step 1; (d) — the hash binding of `H5` / `D3` and the final `H3` arbitration.

3. Pipeline architecture

The pipeline is a chain of modules over the `IDataSource` / `IDataProcessor` interfaces: a source announces `DataReady(take)`; a processor attaches via `Attach`, processes chunks in `ProcessChunk` under `SyncRoot`, publishes output through the byte (`Emit`) or packet (`EmitPacket`) buffer mode, and is itself a source for the next link. `Complete()` is the end-of-input signal with remainder delivery; it is called strictly along the chain from the head.

Invariant J1 (packet indivisibility). The packet output mode never cuts packets at delivery boundaries: `DataReady` carries whole packets once the `BufferSize` threshold is reached.

Invariant J2 (error transparency). On a source failure its `Error ≠ null` and `Completion` ends with that exception; every processor delegates `Error` upward. The error is absorbed by no link.

Invariant J3 (packet assembly from chunks). The accumulator's `_pending` buffer reconstructs packets from arbitrarily bounded chunks; byte loss and duplication at seams are excluded.

4. The Base64 filter

`ByteRangeFilter` is a deterministic mapper `byte → {pass, discard}` over a 256-flag table; for txt input it passes `A-Z U a-z U 0-9 U {+, /}`. Since a packet encodes into exactly 100 characters without `=`, the alphabet without `=` is complete for legitimate input.

Proposition 4.1 (filter harmlessness). The filter removes no character of a legitimate Base64 encoding of packets; it removes only characters outside the alphabet, which never occur in a packet's encoding. ■

5. The sliding window: forward pass

`SlidingWindowScanner(w, h)` with window `w` (100 txt / 75 bin) and handler `h: window → (advance ≥ 1, packet?)`. Forward pass: on failure the advance is 1; on success — `w` (or whatever the handler returns). The scanner retains the entire input `R`; the pass position is `s`; the delivery boundary is `f ≤ s + w`.

Proposition 5.1 (start-coverage completeness). Let an undamaged packet `p` begin at input position `i` and be recognized by the handler. Then a pass checking positions `i, i+1, ..., i+w-1` includes `i` as a window start. Corollary: the step-1 rule on failure guarantees that no packet start is skipped. ■

Proposition 5.2 (desynchronization cost). Re-synchronizing after inserting `g` garbage bytes requires at most `g + w - 1` failed windows; for a txt stream with one spoiled character in a line — up to `w - 1 = 99` failed windows. ■

6. Packet recognition

The accumulator's predicate `Recognizes(p) : IsHeader(p) := Trunc24(SHA-256(p[0..51])) = p[51..75]` — autonomous; for a sector — the disjunction over slots: `num(p) ∈ [0, Tσ)` and `VerifySectorPacket(p, H5σ)`.

Proposition 6.1 (false-acceptance probabilities). For a random 75-byte candidate: a header — 2^{-192} ; a sector given `H5` — 2^{-72} ; for a stream of `L` packets the combined estimate is $\approx L \cdot 2^{-72}$. ■

Proposition 6.2 (sector dependence on the header). The sector check is computable only given `H5`; `H5` is not derivable from the sectors (SHA-256 strength). Hence, before a file's first header none of its sectors are recognizable; the problem is solved by rebinding (section 10). ■

7. The accumulator: chunks → packets → slots

`StreamProcessor.ProcessChunk` assembles packets (J3) and classifies them: a header — byte-wise comparison with known slots (a match → copy-counter increment; a newcomer → `ReadFrom`, `H5`, a slot, and the `HeaderAccepted` event outside the lock); a sector — `AddSector(num, payload copy)` into every slot that passed the check. The metrics `FileCount`, `TotalReceived*`, `TotalCollisionSectorCount` are snapshots under a lock, readable concurrently with reception.

Invariant J4 (thread safety). All state mutations happen under `SyncRoot` / `_state`; events to outside observers are raised outside the lock.

8. Reception slots and versions

`ReceptionSlot` is the state of one file: the header, its copy counter, and the "number → versions with counters" dictionary. The sorting and accumulation invariants are in `AssemblyGuide.Academic.en.md` (sections 3–4). Within the pipeline the slot is a passive receiver; all decisions belong to the accumulator's classification.

9. Multi-file support

There are as many slots as there are distinct headers. A sector is checked against all slots; by Proposition 6.1 an intersection (a sector valid for two different `H5` s) is practically impossible.

Proposition 9.1 (file independence). Receiving one file does not affect another's counters; a rebinding for header `f` checks membership in `f` only. ■

10. Targeted rebinding

On `HeaderAccepted(header f)` the decoder requests of both scanners a re-pass over the retained data with the predicate `RebindWindow`: decode the Base64 window (txt), the number within `[0, T_f)`, `VerifySectorPacket(·, H5_f)`. The active scanner executes the request deferred with boundary `f_pos` (the forward pass's delivery position at request time); an idle one — immediately over the whole traversed extent, with the accumulator temporarily reattached.

Proposition 10.1 (rebinding completeness). Every undamaged sector of file `f` lying in the retained data below the rebinding boundary will be recognized by the re-pass (by Proposition 5.1 applied to the exhaustive position enumeration). ■

11. The rescan

Two properties of the re-pass. **Exhaustiveness:** the window is checked at every position of the region `[0, bound)` — with no jumps; this closes the forward pass's omissions, where a post-success jump of `w` could leap over the start of an overlapping packet. **The boundary** `f` is fixed in `OnDelivering` before the `DataReady` handlers run; re-passing the region beyond `f` is forbidden.

Proposition 11.1 (no duplication). An input byte participates in a re-pass delivery under a given predicate at most once: the boundary is monotone in the forward pass's deliveries and is never extended by re-passes. Corollary: a version's confirmation count does not exceed the number of the packet's physical copies in the input. ■

Cost estimate. A re-pass costs $O(\text{bound} \cdot w)$ window operations; the scanner's memory is $O(|X|)$.

12. Pipeline finalization

`Complete` is called along the chain: filter → scanner → accumulator. The scanner's final delivery raises `f` and executes the deferred rebinds.

Proposition 12.1 (finalization completeness). After `processor.Complete()`, all data delivered by the forward pass is covered by rebinds for every header accepted by that moment. Skipping finalization violates property 2 of the problem statement. ■

13. Progress and cancellation

Progress is $\text{consumed} \times 100 / |X|$ on the `ConsumedAdvanced` event; the phase is `HeaderSearch` until the first slot, then `SectorSearch`; the terminal report is `Done` (100). Cancellation: the `CancellationToken` is checked in the progress report and while awaiting `Completion`; the exception goes outward and the accumulated state survives (J4).

14. Assembly

`TryAssemble(header)` — serialization, a byte-wise slot search, delegation to `slot.TryAssemble(rs, ...)`. The properties of the result are in `AssemblyGuide.Academic.en.md`.

15. The I/O module and the error channel

The sources `ByteArraySource` / `StreamSource` / `FileSource` inherit `BufferedSourceBase`: a read exception is recorded in `Error`, the source stops, `Completion` faults (J2). The sinks `StreamDataWriter` / `FileDataWriter` / `ByteListWriter` / `PreallocatedBufferWriter` write the assembly result.

16. Summary of guarantees and limits

Invariants: **J1** packet indivisibility; **J2** error transparency; **J3** assembly from chunks; **J4** thread safety.

Guarantees:

1. Recognition accuracy: $2^{-192} / 2^{-72}$ (6.1); substitution requires a preimage of the truncated SHA-256.
2. Reception completeness for undamaged packets under the step-1 rule and the exhaustive re-pass (5.1, 10.1, 12.1).

3. No duplication of confirmations (11.1).
4. Independence of multi-file reception (9.1).
5. Delivery of I/O errors to the calling code (J2).

Limits: $O(|X|)$ memory per scanner (the price of rebinding); a re-pass costs $O(\text{bound} \cdot w)$; rebinding covers only retained data; sectors beyond the forward pass's delivery boundary are picked up only by subsequent deliveries/finalization.

Numeric summary: window 100/75; failure step 1; probabilities $2^{-192}/2^{-72}$; the cost of a spoiled character ≤ 99 windows; buffer delivery thresholds 1200–4096 bytes.